

CSEN 296B-2: Week 1.2 Submission 2 — Initial Requirement Set

Student Name: Atishay Jain **Date:** April 1, 2026
Project: AstraNotes
Chosen Technical Path: Python

Architecture Background

Following the architecture work in Submission 1, AstraNotes is designed as a modular Python application with a layered structure: a note data model, an abstract repository for persistence, a file-based storage adapter, a privacy service for encrypted notes, and a validation layer. The requirements below are derived from and intended to support that architecture throughout the quarter.

Functional Requirements

ID	Requirement
FR-01	The system shall enable a user to create a new note by providing a title and body in plain text.
FR-02	The system shall enable a user to modify an existing note's title or body and persist the changes with an automatically updated last-modified timestamp.
FR-03	The system shall enable a user to remove a note from the collection using its unique identifier.
FR-04	The system shall enable a user to flag a note as private, causing its body to be encrypted before it is written to storage.
FR-05	The system shall save all notes to a local file-based store so that data is retained between application sessions.
FR-06	The system shall restore all previously saved notes from local storage on startup.
FR-07	Each note shall carry metadata including a unique identifier (UUID), creation timestamp, last-modified timestamp, and a list of user-assigned tags.
FR-08	The system shall provide keyword-based search across note titles and bodies, returning a list of all matching notes.

Non-Functional Requirements

ID	Requirement
NFR-01	The system shall handle a local collection of up to 500 notes without degradation, completing list and search operations in under 2 seconds.
NFR-02	The codebase shall enforce separation of concerns by organizing source files into distinct layers for models, services, repositories, and application logic.

ID	Requirement
NFR-03	The architecture shall allow individual components (e.g., storage adapter, privacy service) to be tested in isolation through dependency injection or interface substitution.

Security, Privacy, Reliability, and Governance Requirements

ID	Requirement
SPR-01	Notes marked as private shall be encrypted at rest using symmetric encryption (e.g., Fernet from Python's <code>cryptography</code> package) so that raw content is never stored in plaintext.
SPR-02	The system shall catch and handle all storage-related failures (file not found, permission denied, corrupt data) and surface clear, non-technical error messages to the user.
SPR-03	All major components shall be covered by automated tests or a documented manual verification procedure to support quality assurance before each milestone delivery.
SPR-04	Any third-party library introduced into the project shall be documented with its purpose and license; unverified or unnecessary dependencies shall be avoided.

How These Requirements Support the Quarter Project

These 15 requirements are written to serve as a practical foundation for the work ahead:

- **Backlog items:** Each functional requirement (FR-01 through FR-08) maps directly to an implementable user story or development task.
- **Design decisions:** NFR-02 and NFR-03 codify the architectural separation established in Submission 1, preventing drift as the project grows.
- **Prototypes:** The functional requirements define what an early working prototype must demonstrate to be considered viable.
- **Tests:** Every requirement includes a verifiable condition (e.g., "under 2 seconds," "encrypted at rest," "non-technical error messages") that translates into a concrete test case.
- **Acceptance criteria:** The specificity of each requirement makes it possible to determine pass or fail at review time without ambiguity.

Reflection

The first set of AI-generated requirements was too broad to be useful — items like "the app should let users take notes" are not actionable. By pushing the AI to produce requirements that were testable, measurable, and categorized across functional, non-functional, and governance concerns, I arrived at a set that can actually drive development decisions. The exercise reinforced that requirements engineering is not about listing features — it is about defining constraints precise enough to build, test, and defend against.